

GalaxyPets

PHP → Blazor Web App

Migration & Rebuild Guide

For the 3-person team: complete architecture reference, file mapping, task division, database design, and Blazor patterns.

1. Project Overview

GalaxyPets is a browser-based virtual pet social world in the style of Club Penguin and Animal Jam. Players adopt pets, customise them with clothing items from the shop, join clubs, play mini-games, and chat with other players in real time.

The project is being migrated from a hand-rolled PHP stack to ASP.NET Core Blazor Web App with SignalR for real-time features and Dapper for database access.

1.1 Why Blazor Web App

- Write almost everything in C# — very little JavaScript needed
- Interactive components handle clicks, forms, and live updates without page reloads
- SignalR integration is first-class and built in
- Razor syntax (.razor files) feels similar to the PHP/HTML mixing the team already knows
- wwwroot, appsettings.json, and Services stay identical to what MVC would use

1.2 Important: Interactivity Mode Decision

⚠ Before writing any components, the team must agree on one interactivity mode. This affects every .razor file.

Mode	What it means for GalaxyPets
Interactive Server (recommended)	C# runs on the server, UI updates over a SignalR connection. Easiest to start, works great for chat and real-time features. Requires persistent server connection.
Interactive WebAssembly	C# compiles to WASM and runs in the browser. Slower initial load, DB calls must go through an API. More complex.
Interactive Auto	Starts as Server, switches to WASM after download. Most complex, avoid for now.
None (Static SSR)	No interactivity — forms do full page reloads. Too limited for a game world.

✓ **TIP: Use Interactive Server for this project. It is the closest to how PHP worked (server processes everything) and makes the chatroom trivial with SignalR.**

To confirm or set Interactive Server, check Program.cs for:

```
builder.Services.AddRazorComponents()  
    .AddInteractiveServerComponents();
```

And in App.razor or Routes.razor:

```
<Routes @rendermode="InteractiveServer" />
```

2. Previous Architecture (PHP)

The original project used a flat PHP file-per-page approach with no framework. Each page was a self-contained script mixing HTML, business logic, and SQL queries.

2.1 Old Folder Structure

```

GalaxyPetsPHP/
  index.php           -- home page
  login.php           -- login form + processing
  logout.php          -- session destroy + redirect
  register.php        -- registration form + processing
  send_message.php    -- global messaging
  config.php          -- DB credentials
  functions.php       -- shared utility functions
Pages/
  activeusers.php     -- online user list
  adopt.php           -- pet adoption list
  adoption.php        -- confirm adoption
  closet.php          -- avatar clothing editor
  clubs.php           -- clubs list
  games.php           -- games list
  petprofile.php      -- individual pet profile (?id=x)
  shop.php            -- item shop
Games/
  2048Game.php
  flappyalien.php
  matching.php
  snake.php
C4/
clubs/
  AtomsHalo.php
  GalaxyGlitter.php
  Orbitors.php
  ZeroGravity.php
  chatroom.php       -- live chat (polling)
Assets/
  css/               -- per-page stylesheets + pixel.ttf + planet.TTF
  html/navbar.php    -- shared navigation include
  img/               -- boarders, cursor, glorbis, hats, jelly, planets
  js/                -- 2048.js, clubpage.js, flappyalien.js,
                       matching.js, navbar.js, petprofile.js, snake.js
  json/items.json
  planets.json
  php/               -- AJAX handler scripts
    login_handler.php
    buy_item.php
    closet_handler.php
    shop_load_handler.php
    scoreADder.php
    scoreSUBTACTer.php

```

2.2 Old Stack Summary

Component	Technology
Language	PHP 7/8
Routing	File-per-page (no framework)
Templating	Inline PHP/HTML mixed in each file
Database	Raw SQL via mysqli/PDO
Real-time chat	Polling (fetch every N seconds)
Email	PHPMailer
Auth	PHP sessions (\$_SESSION)

Static assets	Assets/ folder served directly
---------------	--------------------------------

3. New Architecture (Blazor Web App)

The new project uses ASP.NET Core Blazor Web App. Pages are .razor components that combine UI and logic. The layout, services, and database layers are shared C# code.

3.1 The Core Mental Model Shift

⚠ This is the most important thing to understand before writing any code.

PHP concept	Blazor equivalent
Each .php file is a page	Each .razor file in Components/Pages/ is a page
HTML + PHP mixed in one file	HTML + C# mixed in one .razor file (@code block at bottom)
include 'navbar.php'	MainLayout.razor wraps every page automatically
\$_SESSION['user']	AuthenticationStateProvider — injected into components
\$_POST['field']	Two-way bound property: @bind="myField" in the form
\$_GET['id']	Route parameter: @page "/pet/{Id:int}" then [Parameter] public int Id
header('Location: x')	NavigationManager.NavigateTo("/path")
fetch('buy_item.php')	Call a C# method directly — no HTTP needed
echo ' Hello '	Just write @myVariable in the razor markup
PHPMailer	MailKit NuGet package

3.2 Anatomy of a .razor File

Every page in Blazor is a .razor file. Here is the structure, using shop.php as an example:

```
@page "/"shop"
@rendermode InteractiveServer
@inject ShopService ShopSvc
@inject AuthenticationStateProvider Auth

<h1>Shop</h1>
@foreach (var item in items)
{
    <div class="item-card">
        
        <p>@item.Name — @item.Price coins</p>
        <button @onclick="() => BuyItem(item.Id)">Buy</button>
    </div>
}

@code {
    private List<Item> items = new();
```

```
protected override async Task OnInitializedAsync()
{
    items = await ShopSvc.GetItemsAsync();
}

private async Task BuyItem(int itemId)
{
    await ShopSvc.BuyItemAsync(itemId);
}
}
```

NOTE: The @code block at the bottom replaces both the PHP handler file AND the controller. No separate file needed.

3.3 New Folder Structure

```
GalaxyPets/ (Blazor Web App project root)
Components/
  Layout/
    MainLayout.razor      -- replaces navbar.php (wraps all pages)
    NavMenu.razor         -- navigation links
  Pages/
    Home.razor            -- index.php
  Account/
    Login.razor           -- login.php + login_handler.php
    Register.razor        -- register.php
    Logout.razor          -- logout.php
  Users/
    ActiveUsers.razor     -- activeusers.php
  Adopt/
    Adopt.razor           -- adopt.php
    Adoption.razor        -- adoption.php
  Closet/
    Closet.razor          -- closet.php + closet_handler.php
  Clubs/
    Clubs.razor           -- clubs.php
    AtomsHalo.razor        -- clubs/AtomsHalo.php
    GalaxyGlitter.razor    -- clubs/GalaxyGlitter.php
    Orbitors.razor         -- clubs/Orbitors.php
    ZeroGravity.razor      -- clubs/ZeroGravity.php
    Chatroom.razor         -- clubs/chatroom.php (UPGRADED: SignalR)
  Games/
    Games.razor           -- games.php
    Game2048.razor         -- 2048Game.php
    FlappyAlien.razor      -- flappyalien.php
    Matching.razor         -- matching.php
    Snake.razor           -- snake.php
    C4.razor              -- C4/
  Pet/
    PetProfile.razor       -- petprofile.php
  Shop/
    Shop.razor             -- shop.php + shop_load_handler.php + buy_item.php
  App.razor               -- root component
  Routes.razor            -- routing config
Hubs/
  ChatHub.cs              -- SignalR hub for chatroom
Models/
  Pet.cs
  User.cs
  Item.cs
  Club.cs
  Score.cs
Services/
  AuthService.cs          -- login/register logic
```

```

ShopService.cs      -- buy_item + shop_load logic
ClosetService.cs    -- closet_handler logic
GameService.cs      -- functions.php + score handlers
PetService.cs       -- adopt + profile logic
wwwroot/
css/                -- ALL from Assets/css/ (direct copy)
fonts/              -- pixel.ttf, planet.TTF
img/                -- ALL from Assets/img/ (direct copy)
js/                 -- ALL from Assets/js/ (direct copy)
json/               -- items.json, planets.json
appsettings.json    -- DB connection string (replaces config.php)
Program.cs          -- app bootstrap, DI, SignalR, auth

```

3.4 New Stack Summary

Component	Technology
Language	C# 12 / .NET 8
Framework	ASP.NET Core Blazor Web App
Pages	.razor components in Components/Pages/
Interactivity	Interactive Server (SignalR-backed)
Database	Dapper (raw SQL — no ORM learning curve)
Real-time chat	SignalR ChatHub.cs (first-class in Blazor)
Email	MailKit (replaces PHPMailer)
Auth	ASP.NET Core Cookie Auth + AuthenticationStateProvider
Static assets	wwwroot/ — identical to MVC
IDE	VS Code with C# Dev Kit extension

4. Complete File Migration Map

4.1 Root PHP Files

PHP file	Blazor component	Notes
index.php	Components/Pages/Home.razor	@page "/"
login.php + login_handler.php	Components/Pages/Account/Login.razor	Form + handler merged in @code block
logout.php	Components/Pages/Account/Logout.razor	Call AuthService.Logout() then NavigateTo
register.php	Components/Pages/Account/Register.razor	Form + handler in @code block
send_message.php	Services/MessageService.cs	Called directly from any component
config.php	appsettings.json	ConnectionStrings section

functions.php	Services/GameService.cs	Static utility methods in C#
Assets/html/navbar.php	Components/Layout/MainLayout.razor	Wraps all pages automatically

4.2 Pages/ Folder

PHP file	Blazor component	Route
Pages/activeusers.php	Pages/Users/ActiveUsers.razor	@page "/users/active"
Pages/adopt.php	Pages/Adopt/Adopt.razor	@page "/adopt"
Pages/adoption.php	Pages/Adopt/Adoption.razor	@page "/adopt/{Id:int}"
Pages/closet.php + closet_handler.php	Pages/Closet/Closet.razor	@page "/closet"
Pages/clubs.php	Pages/Clubs/Clubs.razor	@page "/clubs"
Pages/games.php	Pages/Games/Games.razor	@page "/games"
Pages/petprofile.php?id=x	Pages/Pet/PetProfile.razor	@page "/pet/{Id:int}"
Pages/shop.php + shop_load_handler.php + buy_item.php	Pages/Shop/Shop.razor	@page "/shop"

4.3 Games/

PHP file	Blazor component	Notes
Games/2048Game.php	Pages/Games/Game2048.razor	Renders <canvas>, loads wwwroot/js/2048.js via JS interop
Games/flappyalien.php	Pages/Games/FlappyAlien.razor	Loads wwwroot/js/flappyalien.js
Games/matching.php	Pages/Games/Matching.razor	Loads wwwroot/js/matching.js
Games/snake.php	Pages/Games/Snake.razor	Loads wwwroot/js/snake.js
Games/C4/	Pages/Games/C4.razor	Loads C4 JS from wwwroot/js/c4/
Games/blue.png + spaceship.webp	wwwroot/img/games/	Direct copy, no component needed
scoreADder.php + scoreSUBTACTer.php	Services/GameService.cs AddScore / SubtractScore	Called from game components

4.4 Clubs/

PHP file	Blazor component	Notes
clubs/AtomsHalo.php	Pages/Clubs/AtomsHalo.razor	@page "/clubs/atomshalo"
clubs/GalaxyGlitter.php	Pages/Clubs/GalaxyGlitter.razor	@page "/clubs/galaxyglitter"
clubs/Orbitors.php	Pages/Clubs/Orbitors.razor	@page "/clubs/orbitors"
clubs/ZeroGravity.php	Pages/Clubs/ZeroGravity.razor	@page "/clubs/zerogravity"

clubs/chatroom.php	Pages/Clubs/Chatroom.razor	UPGRADE: replace polling with SignalR — see Section 7
--------------------	----------------------------	---

4.5 Assets/ to wwwroot/ (Direct Copy)

PHP asset path	Blazor wwwroot path
Assets/css/*.css	wwwroot/css/*.css — copy unchanged
Assets/css/pixel.ttf	wwwroot/fonts/pixel.ttf
Assets/css/planet.TTF	wwwroot/fonts/planet.TTF
Assets/img/boarders/	wwwroot/img/boarders/ — copy unchanged
Assets/img/cursor/	wwwroot/img/cursor/ — copy unchanged
Assets/img/glorbus/	wwwroot/img/glorbus/ — copy unchanged
Assets/img/hats/	wwwroot/img/hats/ — copy unchanged
Assets/img/jelly/	wwwroot/img/jelly/ — copy unchanged
Assets/img/planets/	wwwroot/img/planets/ — copy unchanged
Assets/img/aquagalaxy_small.gif	wwwroot/img/aquagalaxy_small.gif
Assets/js/2048.js	wwwroot/js/2048.js — copy unchanged
Assets/js/clubpage.js	wwwroot/js/clubpage.js — copy unchanged
Assets/js/flappyalien.js	wwwroot/js/flappyalien.js — copy unchanged
Assets/js/matching.js	wwwroot/js/matching.js — copy unchanged
Assets/js/navbar.js	wwwroot/js/navbar.js — copy unchanged
Assets/js/petprofile.js	wwwroot/js/petprofile.js — copy unchanged
Assets/js/snake.js	wwwroot/js/snake.js — copy unchanged
Assets/json/items.json	wwwroot/json/items.json (seed to DB later)
Assets/json/planets.json	wwwroot/json/planets.json (seed to DB later)

4.6 PHP Handler Files (Biggest Change in Blazor)

In PHP, AJAX calls hit separate handler .php files. In Blazor, these handlers simply become methods inside the relevant component's @code block or in a Service class. No HTTP call needed.

PHP handler	Blazor equivalent
Assets/php/login_handler.php	Login() method in Login.razor @code block — calls AuthService.LoginAsync()
Assets/php/buy_item.php	BuyItem(int id) method in Shop.razor @code block — calls ShopService.BuyAsync()
Assets/php/closet_handler.php	SaveCloset() method in Closet.razor @code block — calls ClosetService.SaveAsync()

Assets/php/shop_load_handler.php	OnInitializedAsync() in Shop.razor — calls ShopService.GetItemsAsync()
Assets/php/scoreADDER.php	AddScore() in game .razor @code block — calls GameService.AddScoreAsync()
Assets/php/scoreSUBTRACTer.php	SubtractScore() in game .razor @code block — calls GameService.SubtractScoreAsync()

5. Database Design

The PHP version used raw SQL but no documented schema. Below is the recommended schema for the Blazor version based on all features observed in the PHP codebase. Use SQL Server (LocalDB for dev) or PostgreSQL — both work with Dapper.

⚠ Compare each table against your existing PHP database before running CREATE TABLE. Column names may differ. Do not drop your old DB until migration is verified.

5.1 Tables

Users

```
CREATE TABLE Users (
  Id          INT IDENTITY PRIMARY KEY,
  Username    NVARCHAR(50)  NOT NULL UNIQUE,
  Email       NVARCHAR(100) NOT NULL UNIQUE,
  PasswordHash NVARCHAR(256) NOT NULL,
  Coins       INT           NOT NULL DEFAULT 0,
  IsOnline    BIT           NOT NULL DEFAULT 0,
  LastSeen    DATETIME2     NOT NULL DEFAULT GETDATE(),
  CreatedAt   DATETIME2     NOT NULL DEFAULT GETDATE()
);
```

Pets

```
CREATE TABLE Pets (
  Id          INT IDENTITY PRIMARY KEY,
  OwnerId     INT           NOT NULL REFERENCES Users(Id),
  Name        NVARCHAR(50)  NOT NULL,
  Species     NVARCHAR(50)  NOT NULL,
  PlanetId    INT           NULL REFERENCES Planets(Id),
  CreatedAt   DATETIME2     NOT NULL DEFAULT GETDATE()
);
```

Items (shop catalogue — seed from items.json)

```
CREATE TABLE Items (
  Id          INT IDENTITY PRIMARY KEY,
  Name        NVARCHAR(100) NOT NULL,
  Category    NVARCHAR(50)  NOT NULL, -- hat, border, etc.
  Price       INT           NOT NULL,
  ImagePath   NVARCHAR(200) NOT NULL
);
```

Inventory (items owned by a pet)

```
CREATE TABLE Inventory (
  Id          INT IDENTITY PRIMARY KEY,
  PetId       INT NOT NULL REFERENCES Pets(Id),
  ItemId      INT NOT NULL REFERENCES Items(Id),
```

```
    Equipped BIT NOT NULL DEFAULT 0
);
```

Planets (seed from planets.json)

```
CREATE TABLE Planets (
    Id          INT IDENTITY PRIMARY KEY,
    Name        NVARCHAR(100) NOT NULL,
    ImagePath   NVARCHAR(200) NOT NULL
);
```

Clubs

```
CREATE TABLE Clubs (
    Id          INT IDENTITY PRIMARY KEY,
    Slug        NVARCHAR(50)  NOT NULL UNIQUE,
    DisplayName NVARCHAR(100) NOT NULL
);
```

ClubMembers

```
CREATE TABLE ClubMembers (
    UserId INT NOT NULL REFERENCES Users(Id),
    ClubId INT NOT NULL REFERENCES Clubs(Id),
    PRIMARY KEY (UserId, ClubId)
);
```

ChatMessages

```
CREATE TABLE ChatMessages (
    Id          INT IDENTITY PRIMARY KEY,
    ClubId      INT          NOT NULL REFERENCES Clubs(Id),
    UserId      INT          NOT NULL REFERENCES Users(Id),
    Message     NVARCHAR(500) NOT NULL,
    SentAt      DATETIME2    NOT NULL DEFAULT GETDATE()
);
```

Scores

```
CREATE TABLE Scores (
    Id          INT IDENTITY PRIMARY KEY,
    UserId      INT          NOT NULL REFERENCES Users(Id),
    GameSlug    NVARCHAR(50) NOT NULL, -- 'snake', '2048', 'flappy', etc.
    Score       INT          NOT NULL,
    AwardedAt   DATETIME2    NOT NULL DEFAULT GETDATE()
);
```

Seed clubs table

```
INSERT INTO Clubs (Slug, DisplayName) VALUES
    ('atomshalo',    'Atoms Halo'),
    ('galaxyglitter','Galaxy Glitter'),
    ('orbitors',     'Orbitors'),
    ('zerogravity',  'Zero Gravity');
```

6. Program.cs Setup

This must be configured correctly before any components work. Set this up as a team during Phase 0.

```
var builder = WebApplication.CreateBuilder(args);

// Blazor + Interactive Server
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();
```

```
// SignalR (for ChatHub)
builder.Services.AddSignalR();

// Dapper - DB connection
builder.Services.AddScoped<IDbConnection>(_ =>
    new SqlConnection(
        builder.Configuration.GetConnectionString("DefaultConnection")));

// Services
builder.Services.AddScoped<AuthService>();
builder.Services.AddScoped<ShopService>();
builder.Services.AddScoped<ClosetService>();
builder.Services.AddScoped<GameService>();
builder.Services.AddScoped<PetService>();

// Cookie auth
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(opt => { opt.LoginPath = "/account/login"; });
builder.Services.AddCascadingAuthenticationState();
builder.Services.AddAuthorizationCore();

var app = builder.Build();
app.UseStaticFiles();
app.UseAuthentication();
app.UseAuthorization();
app.UseAntiforgery();

app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();

// SignalR hub
app.MapHub<ChatHub>("/chathub");

app.Run();
```

NOTE: Add connection string in appsettings.json: { "ConnectionStrings": { "DefaultConnection": "Server=(localdb)\mssqllocaldb;Database=GalaxyPets;Trusted_Connection=True" } }

7. Using Your Existing JS Games in Blazor

Your game JS files (snake.js, 2048.js, flappyalien.js, matching.js) copy to wwwroot/js/ unchanged. Blazor needs a small JS interop call to initialise them after the component renders.

7.1 Game Component Pattern (same for all games)

Example: Snake.razor

```
@page "/games/snake"
@rendermode InteractiveServer
@inject IJSRuntime JS

<h2>Snake</h2>
<canvas id="snakeCanvas" width="400" height="400"></canvas>

@code {
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
```

```
        if (firstRender)
            await JS.InvokeVoidAsync("initSnake", "snakeCanvas");
    }
}
```

NOTE: OnAfterRenderAsync with firstRender=true is the Blazor equivalent of window.onload. The JS file must expose an init function, e.g. window.initSnake = function(canvasId) { ... }

Add the script reference in App.razor or the layout, just before </body>:

```
<script src="js/snake.js"></script>
```

7.2 Score Submission from JS to Blazor

If a game needs to submit a score when it ends, use `DotNet.invokeMethodAsync` from the JS side:

```
// In snake.js, when game over:
DotNet.invokeMethodAsync('GalaxyPets', 'SubmitScore', finalScore);
```

And in the .razor `@code` block:

```
[JSInvokable]
public static async Task SubmitScore(int score)
{
    // save to DB via GameService
}
```

8. SignalR Chatroom (Replacing chatroom.php)

The chatroom upgrade is the most impactful change. PHP was using polling (fetch every few seconds). SignalR gives true real-time messaging with no polling.

8.1 Server-side Hub (Hubs/ChatHub.cs)

```
public class ChatHub : Hub
{
    public async Task SendMessage(string clubSlug, string message)
    {
        var user = Context.User?.Identity?.Name ?? "Guest";
        await Clients.Group(clubSlug)
            .SendAsync("ReceiveMessage", user, message);
    }

    public async Task JoinClub(string clubSlug)
    => await Groups.AddToGroupAsync(Context.ConnectionId, clubSlug);
}
```

8.2 Chatroom.razor (Full Component)

Because Blazor uses Interactive Server mode, the chatroom component can use SignalR directly in C# — no JavaScript needed for the chat logic at all.

```
@page "/clubs/chatroom/{ClubSlug}"
@rendermode InteractiveServer
@inject NavigationManager Nav
@inject AuthenticationStateProvider AuthProvider
@implements IAsyncDisposable
```

```
@using Microsoft.AspNetCore.SignalR.Client

<h2>@ClubSlug Chatroom</h2>

<div class="chat-box">
    @foreach (var msg in messages)
    {
        <p><strong>@msg.User:</strong> @msg.Text</p>
    }
</div>

<input @bind="currentMessage" placeholder="Say something..." />
<button @onclick="Send">Send</button>

@code {
    [Parameter] public string ClubSlug { get; set; } = "";
    private HubConnection? hub;
    private List<(string User, string Text)> messages = new();
    private string currentMessage = "";

    protected override async Task OnInitializedAsync()
    {
        hub = new HubConnectionBuilder()
            .WithUrl(Nav.ToAbsoluteUri("/chathub"))
            .Build();

        hub.On<string, string>("ReceiveMessage", (user, msg) => {
            messages.Add((user, msg));
            InvokeAsync(StateHasChanged);
        });

        await hub.StartAsync();
        await hub.SendAsync("JoinClub", ClubSlug);
    }

    private async Task Send()
    {
        if (!string.IsNullOrWhiteSpace(currentMessage))
        {
            await hub!.SendAsync("SendMessage", ClubSlug, currentMessage);
            currentMessage = "";
        }
    }

    public async ValueTask DisposeAsync()
        => await (hub?.DisposeAsync() ?? ValueTask.CompletedTask);
}
```

✓ **TIP: No JavaScript at all for the chat — the C# SignalR client handles everything. This is a major Blazor advantage over MVC.**

9. Work Division (3-Person Team)

Suggested split based on feature groupings. Complete Phase 0 together first — everyone needs auth and the DB working before building pages.

⚠ Do Phase 0 as a team before splitting. Nothing works without a connected DB and working auth.

Phase 0 — Do Together First (Est. 1-2 days)

1. Confirm Interactive Server render mode in Program.cs and App.razor
2. Set up appsettings.json with DB connection string
3. Run CREATE TABLE SQL scripts (Section 5) against your database
4. Configure Program.cs — auth, SignalR, Dapper, services (Section 6)
5. Copy ALL wwwroot assets from Assets/ folder (Section 4.5) — straight copy
6. Build MainLayout.razor + NavMenu.razor from navbar.php
7. Test that the project runs and serves the home page with correct CSS

Person 1 — Auth, Profiles & Social

Focus: login/register, pet profiles, active users, messaging, adoption

- Components/Pages/Account/Login.razor — port login.php + login_handler.php
- Components/Pages/Account/Register.razor — port register.php
- Components/Pages/Account/Logout.razor — call AuthService, NavigateTo /
- Services/AuthService.cs — LoginAsync(), RegisterAsync(), LogoutAsync()
- Components/Pages/Pet/PetProfile.razor — port petprofile.php (@page "/pet/{Id:int}")
- Components/Pages/Users/ActiveUsers.razor — port activeusers.php
- Services/MessageService.cs — port send_message.php logic
- Components/Pages/Adopt/Adopt.razor — port adopt.php
- Components/Pages/Adopt/Adoption.razor — port adoption.php
- Services/PetService.cs — adoption and profile DB queries
- MailKit setup — replace PHPMailer for registration confirmation emails

Person 2 — Shop, Closet & Economy

Focus: shop display, buying items, closet/avatar editor, coin system, home page

- Components/Pages/Shop/Shop.razor — port shop.php + shop_load_handler.php + buy_item.php
- Services/ShopService.cs — GetItemsAsync(), BuyAsync(userId, itemId)
- Components/Pages/Closet/Closet.razor — port closet.php + closet_handler.php
- Services/ClosetService.cs — GetEquippedAsync(), SaveAsync()
- Components/Pages/Home.razor — port index.php
- Seed items.json into Items table via a one-time seed script
- Seed planets.json into Planets table
- Models/Item.cs, Models/Pet.cs, Models/User.cs — C# model classes

Person 3 — Games, Clubs & Real-time Chat

Focus: all game pages, all club pages, SignalR chatroom

- Hubs/ChatHub.cs — SignalR hub (Section 8.1)
- Components/Pages/Clubs/Chatroom.razor — full component with C# SignalR client (Section 8.2)
- Components/Pages/Clubs/Clubs.razor — clubs list
- Components/Pages/Clubs/AtomsHalo.razor, GalaxyGlitter.razor, Orbitors.razor, ZeroGravity.razor
- Components/Pages/Games/Games.razor — games list
- Components/Pages/Games/Game2048.razor — canvas + JS interop pattern (Section 7)

- Components/Pages/Games/FlappyAlien.razor, Matching.razor, Snake.razor, C4.razor
- Services/GameService.cs — AddScoreAsync(), SubtractScoreAsync(), port functions.php

10. Gaps & Next Steps After Migration

10.1 Must-Do Before Launch

8. Password hashing — if PHP used MD5 or SHA1, DO NOT migrate those hashes. Use ASP.NET Core PasswordHasher<T> or BCrypt.Net-Next. Force a password reset if needed.
9. CSRF / Antiforgery — add `@attribute [ValidateAntiForgeryToken]` on form-submitting components. Blazor's `UseAntiforgery()` in `Program.cs` handles this globally.
10. Auth guards — add `@attribute [Authorize]` at the top of any component that requires login (Closet, Shop, Pet profile etc.)
11. URL change — PHP used `?id=5`. Blazor uses `/pet/5`. Update any hardcoded links in ported views.
12. Seed data — run `items.json` and `planets.json` seed scripts before shop/adoption go live.

10.2 Upgrades Worth Adding

- Real-time online user count on the home page — use SignalR `OnConnectedAsync/OnDisconnectedAsync` in `ChatHub`
- Leaderboard page using the `Scores` table
- Club membership join/leave buttons wired to `ClubMembers` table
- Toast notifications when someone buys an item or earns a score — trivial with Blazor state
- Error boundary components — wrap pages in `<ErrorBoundary>` to catch crashes gracefully

10.3 NuGet Packages to Install

Package	Purpose
Dapper	Raw SQL — replaces <code>mysqli</code> . <code>dotnet add package Dapper</code>
Microsoft.Data.SqlClient	SQL Server driver. <code>dotnet add package Microsoft.Data.SqlClient</code>
MailKit	Email — replaces <code>PHPMailer</code> . <code>dotnet add package MailKit</code>
BCrypt.Net-Next	Password hashing. <code>dotnet add package BCrypt.Net-Next</code>
Microsoft.AspNetCore.SignalR	Already included in .NET 8 Blazor — no install needed

NOTE: Run all `dotnet add package` commands from the **GalaxyPets** project folder in the terminal.

11. Quick Reference Cheat Sheet

Razor (.razor) Basics

```

@page "/shop"                -- sets the URL route
@rendermode InteractiveServer  -- enables click handlers etc.
@inject ShopService Svc       -- inject a service (like PHP include)
@attribute [Authorize]        -- require login for this page

@* This is a Razor comment *@
@Model.PropertyName           -- output a value (auto HTML-encoded)
@((MarkupString)rawHtml)      -- output raw HTML (use carefully)
@if (condition) { <p>shown</p> }
@foreach (var x in list) { <li>@x.Name</li> }

<input @bind="myField" />      -- two-way bind (replaces $_POST)
<button @onclick="MyMethod">   -- click handler (replaces fetch to .php)
   -- tilde = wwwroot root

```

Common Blazor Patterns

Task	Blazor pattern
Run code on page load	override async Task OnInitializedAsync()
Run code after HTML renders (for JS)	override async Task OnAfterRenderAsync(bool firstRender)
Redirect to another page	NavigationManager.NavigateTo("/path")
Call JavaScript	await JS.InvokeVoidAsync("functionName", args)
Get current logged-in user	var auth = await AuthProvider.GetAuthenticationStateAsync(); var user = auth.User.Identity?.Name
Force UI refresh	StateHasChanged() — call after async updates
Show/hide element	@if (showPanel) { ... }
Loop over DB results	@foreach (var pet in pets) { @pet.Name }